MAGENTO 2 · EVENT & OBSERVER DEMO

Codilar_EventDemo

How the cart special-price observer works, a file-by-file code explanation, and the questions to expect when presenting this demo.

Module	`app/code/Codilar/EventDemo`
Platform	Magento Open Source 2.4.9 · PHP 8.5
Event	`checkout_cart_product_add_after`
Dispatched by	`Magento\Checkout\Model\Cart::addProduct()`
Observer	`Codilar\EventDemo\Observer\CartSpecialPriceObserver`
Scope	Storefront only (`etc/frontend/events.xml`)
Date	October 2026

1. What the demo does
2. Event & observer fundamentals
3. Request flow
4. Module structure
5. File-by-file code explanation
6. Tested scenarios
7. Install, configure and test
8. Design decisions and pitfalls
9. Known limitations
10. Expected questions and answers

1. What the demo does

When a shopper adds one chosen product to the cart, that cart line is given a special price. Here the product is **Realme XT**: it costs **$349.99** in the catalog but shows **$99.00** in the cart. Every other product keeps its normal price, and the catalog price itself never changes.

Step by step

#	Step	What happens in code
1	The customer clicks **Add to Cart**.	Magento adds the product to the cart as usual, in `Magento\Checkout\Model\Cart::addProduct()`.
2	Magento announces that a product was just added.	At the end of `addProduct()` core calls `$this->_eventManager->dispatch('checkout_cart_product_add_after', ['quote_item' => $result, 'product' => $product])`. This is the **event**.
3	Our observer is listening.	`etc/frontend/events.xml` connects the event to `CartSpecialPriceObserver`, so Magento calls its `execute()` method automatically. This is the **observer**.
4	The observer checks which product it was.	If the product's SKU is not `realme-xt`, it returns immediately and does nothing.
5	It sets a special price on that cart line only.	`setCustomPrice(99)` and `setOriginalCustomPrice(99)` on the quote item. The `Cart::save()` call that follows recalculates the totals and stores the price.

Adapted from the brief. The brief's example was a T-shirt at ₹499 → ₹99. The T-shirt no longer exists in this catalogue and the store currency is US dollars, so the demo uses Realme XT at $349.99 → $99. The SKU and price are settings in `etc/frontend/di.xml`, so they can be changed without touching PHP.

Nothing on the admin side was changed. No admin code, configuration, templates or product data were changed. The special price exists only on the cart line.

2. Event & observer fundamentals

Events and **observers** are Magento's implementation of the publish–subscribe pattern. Code that reaches an interesting moment *dispatches* a named event with some data. Any number of modules can *observe* that event and run their own code, without the dispatching code knowing they exist.

Term	Meaning	In this demo
Event	A named announcement plus a data array, sent with `EventManager::dispatch()`.	`checkout_cart_product_add_after` with `quote_item` and `product`
Observer	A class implementing `Magento\Framework\Event\ObserverInterface` whose `execute(Observer $observer)` runs when the event fires.	`CartSpecialPriceObserver`
events.xml	The configuration linking an event name to observer classes, per area.	`etc/frontend/events.xml`
Observer object	The wrapper passed to `execute()`; `$observer->getEvent()` returns the event and its data.	`getEvent()->getData('product')`

How Magento dispatches an event

This is the real core code from `vendor/magento/framework/Event/Manager.php` (2.4.9). For every observer registered for the event name, it builds an `Event` and an `Observer` wrapper holding the data, then hands them to the invoker.

vendor/magento/framework/Event/Manager.php (excerpt)

```
52     public function dispatch($eventName, array $data = [])
53     {
54         $eventName = $eventName !== null ? mb_strtolower($eventName) : '';
55         \Magento\Framework\Profiler::start('EVENT:' . $eventName, ['group' => 'EVENT', 'name' =>
$eventName]);
56         foreach ($this->_eventConfig->getObservers($eventName) as $observerConfig) {
57             $event = new \Magento\Framework\Event($data);
58             $event->setName($eventName);
59 
60             $wrapper = new Observer();
62             $wrapper->setData(array_merge(['event' => $event], $data));
63 
64             \Magento\Framework\Profiler::start('OBSERVER:' . $observerConfig['name']);
65             $this->_invoker->dispatch($observerConfig, $wrapper);
66             \Magento\Framework\Profiler::stop('OBSERVER:' . $observerConfig['name']);
67         }
68         \Magento\Framework\Profiler::stop('EVENT:' . $eventName);
69     }
```

The invoker (`Event\Invoker\InvokerDefault`) then:

- skips the observer if it is configured with `disabled="true"`;
- gets the observer from the object manager: one **shared** instance by default, or a new instance per call if `shared="false"`;
- calls `execute()` if the class implements `ObserverInterface`. Otherwise it throws a `LogicException` in developer mode and logs a warning in production.

The event used here, at its source

vendor/magento/module-checkout/Model/Cart.php — end of addProduct() (excerpt)

```
419         $this->_eventManager->dispatch(
420             'checkout_cart_product_add_after',
421             ['quote_item' => $result, 'product' => $product]
422         );
423         $this->_checkoutSession->setLastAddedProductId($productId);
424         return $this;
```

- `quote_item` – the cart line (`Magento\Quote\Model\Quote\Item`) returned by `Quote::addProduct()`. It is the parent line, the one the shopper sees.
- `product` – the product that was added.
- The event fires **only after a successful add**. If the product can't be added (e.g. out of stock), core throws before reaching this line.
- Magento's own `Magento_Quote` module also observes this event (`SetBasePriceObserver`). It only fills in a missing base price, so it does not clash with ours.

Kinds of events in Magento

Kind	Example
Explicit events in core code	`checkout_cart_product_add_after`, `sales_order_place_after`, `customer_login`
Model lifecycle events (from `$_eventPrefix`)	`catalog_product_save_after`, `sales_order_save_after`, `customer_delete_before`
Controller events	`controller_action_predispatch`, `controller_action_predispatch_checkout_cart_add`
Custom events	Your own module can call `$eventManager->dispatch('codilar_something', [...])` for others to observe.

Observer vs plugin vs preference

Technique	Use it when
Observer	Magento already dispatches an event at the right moment and you want to react to it. Observers can change objects passed in the event data, but cannot change a method's arguments or return value, or stop it running.
Plugin	You need to change the input or output of one specific public method, or decide whether it runs (see Codilar_CartPlugins).
Preference	You must replace a whole class (e.g. swap `Magento\Theme\Block\Html\Footer` for your own subclass). Only one preference can win, so it is the most conflict-prone option.

3. Request flow

What happens when a shopper clicks **Add to Cart** on Realme XT (Magento 2.4.9, with Codilar_CartPlugins also installed).

POST /checkout/cart/add
Magento\Checkout\Controller\Cart\Add::execute() loads the product and calls the AddProductToCart service

▼

Magento\Checkout\Model\AddProductToCart::execute()
calls `$cart->addProduct($product, $buyRequest)`, then `$cart->save()`

▼ addProduct()

Codilar_CartPlugins: guest check and duplicate check (plugins)
Run first. If either throws, addProduct() never runs and the event is never dispatched.

▼

Original Cart::addProduct()
Quote::addProduct() creates the cart line at the catalog price ($349.99)

▼ dispatch()

Event: checkout_cart_product_add_after
data: `quote_item` = the new cart line, `product` = Realme XT

▼ for each observer registered in the frontend area

Magento_Quote · setBasePriceObserver
fills in a missing base price; nothing to do here

Codilar_EventDemo · CartSpecialPriceObserver
SKU is `realme-xt` → `setCustomPrice(99)`, `setOriginalCustomPrice(99)`. Any other SKU → return.

▼ back in AddProductToCart

Cart::save()
collectTotals() uses the custom price ($99 × qty) → quote saved to the database

▼

Controller
“You added Realme XT to your shopping cart.” → cart shows $99.00

Which storefront actions fire the event

Action	Event fired?
Add to Cart on the product page or a category listing	**Yes** – via `Cart::addProduct()`
Wishlist “Add to Cart”	**Yes** – `Wishlist\Model\Item::addToCart()` calls `Cart::addProduct()`
Changing quantity on the cart page	No – but the custom price is already stored on the line, so it stays $99 each
Reorder from “My Orders”	No – in 2.4.9 reorder adds to the `Quote` directly (`Sales\Model\Reorder\Reorder`), so the line gets the catalog price
Admin order creation, REST and GraphQL carts	No – they do not use `Checkout\Model\Cart`, and our events.xml is storefront-only

4. Module structure

app/code/Codilar/EventDemo/

```
EventDemo/
├── registration.php                    registers the module with Magento
├── etc/
│   ├── module.xml                      module name + load order
│   └── frontend/
│       ├── events.xml                  links the event to the observer (storefront only)
│       └── di.xml                      which SKU, which price
└── Observer/
    └── CartSpecialPriceObserver.php    the code that runs when the event fires
```

File	Responsibility
`registration.php`	Tells Magento that `Codilar_EventDemo` lives in this folder.
`etc/module.xml`	Declares the module and loads it after Checkout and Quote.
`etc/frontend/events.xml`	Subscribes the observer to `checkout_cart_product_add_after`.
`etc/frontend/di.xml`	Passes the target SKU and the special price into the observer's constructor.
`Observer/CartSpecialPriceObserver.php`	Checks the product and sets the cart line's custom price.

A module with an observer needs no plugin, preference, controller, block or database change. Configuration plus one class is enough.

5. File-by-file code explanation

5.1 registration.php

app/code/Codilar/EventDemo/registration.php

```php
<?php
use Magento\Framework\Component\ComponentRegistrar;

ComponentRegistrar::register(ComponentRegistrar::MODULE, 'Codilar_EventDemo', __DIR__);
```

- **Line 4** – `ComponentRegistrar::register()` records that the component of type `MODULE` named `Codilar_EventDemo` is located in `__DIR__` (this folder).
- Composer's autoloader loads every `registration.php` under `app/code` at bootstrap. This is how Magento discovers the module.
- The name must follow `Vendor_Module` and match `etc/module.xml`.

5.2 etc/module.xml

app/code/Codilar/EventDemo/etc/module.xml

```xml
<?xml version="1.0"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="urn:magento:framework:Module/etc/module.xsd">
    <module name="Codilar_EventDemo">
        <sequence>
            <module name="Magento_Checkout"/>
            <module name="Magento_Quote"/>
        </sequence>
    </module>
</config>
```

- **Line 4** – the module name. After `bin/magento module:enable` it appears in `app/etc/config.php`.
- **Lines 5–8** – `<sequence>` sets **load order**. Our configuration is merged after `Magento_Checkout` (which dispatches the event) and `Magento_Quote` (the quote item we modify, and the other observer on this event).
- No `setup_version`: the module has no database schema or data.

5.3 etc/frontend/events.xml

app/code/Codilar/EventDemo/etc/frontend/events.xml

```xml
<?xml version="1.0"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="urn:magento:framework:Event/etc/events.xsd">
    <!-- A core event, dispatched in Magento\Checkout\Model\Cart::addProduct() right after a
         product goes into the cart, with the data ['quote_item' => $item, 'product' => $product].
         Declared in etc/frontend so the admin is not affected.
         Add the product below to the cart and look at its price. -->
    <event name="checkout_cart_product_add_after">
        <observer name="codilar_eventdemo_cart_special_price"
                  instance="Codilar\EventDemo\Observer\CartSpecialPriceObserver"/>
    </event>
</config>
```

- **Location** – the file is in `etc/frontend/`, so the observer is registered only in the storefront area. A file in `etc/` would apply to every area (admin, REST, GraphQL, cron…). Other area folders include `etc/adminhtml/`, `etc/webapi_rest/` and `etc/graphql/`.

- **Line 3** – the XSD (`Event/etc/events.xsd`) lets Magento and IDEs validate the file.
- **Lines 4–7** – a comment recording where the event is dispatched and what data it carries. Event data is not type-checked, so this documentation saves the next developer from searching the core code.
- **Line 8** – `<event name="…">` must exactly match the name used in `dispatch()`. Event names are lower-case with underscores.
- **Lines 9–10** – `<observer>`:
  - `name` – unique per event. Another module can use the same event + name with `disabled="true"` to switch it off.
  - `instance` – the observer class, created by the object manager (so constructor injection works).
  - Optional attributes not used here: `disabled="true|false"` and `shared="false"` (a new instance per dispatch instead of one shared instance).

5.4 etc/frontend/di.xml

app/code/Codilar/EventDemo/etc/frontend/di.xml

```xml
<?xml version="1.0"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="urn:magento:framework:ObjectManager/etc/config.xsd">
    <!-- Which product gets the cart-only price, and what that price is. -->
    <type name="Codilar\EventDemo\Observer\CartSpecialPriceObserver">
        <arguments>
            <argument name="sku" xsi:type="string">realme-xt</argument>
            <argument name="specialPrice" xsi:type="number">99</argument>
        </arguments>
    </type>
</config>
```

- **Line 5** – `<type name="…">` configures how the object manager builds our observer.
- **Lines 6–9** – `<arguments>` are matched to constructor parameters **by name**: `$sku` receives `realme-xt` and `$specialPrice` receives `99`. `xsi:type` says how to interpret the value (`string`, `number`, `boolean`, `object`, `array`…).
- Keeping the values here means a different product or price is a one-line XML change plus `cache:flush`, with no PHP change. It is also in `etc/frontend`, matching where the observer is used.

5.5 Observer/CartSpecialPriceObserver.php

app/code/Codilar/EventDemo/Observer/CartSpecialPriceObserver.php

```php
<?php
declare(strict_types=1);

namespace Codilar\EventDemo\Observer;

use Magento\Catalog\Model\Product;
use Magento\Framework\Event\Observer;
use Magento\Framework\Event\ObserverInterface;
use Magento\Quote\Model\Quote\Item;

/**
 * Listens to "checkout_cart_product_add_after", which Magento fires every time a product
 * is added to the cart. If it is the chosen product (Realme XT, normally $349.99), that
 * cart line gets a special price ($99). The catalog price itself is not changed.
 */
class CartSpecialPriceObserver implements ObserverInterface
{
```

```
18      public function __construct(
19          private readonly string $sku,
20          private readonly float $specialPrice
21      ) {
22      }
23
24      public function execute(Observer $observer): void
25      {
26          /** @var Product $product */
27          $product = $observer->getEvent()->getData('product');
28          if ($product->getSku() !== $this->sku) {
29              return;
30          }
31
32          /** @var Item $item */
33          $item = $observer->getEvent()->getData('quote_item');
34
35          // Custom price applies to this cart line only. Cart::save(), which runs right after,
36          // recalculates the totals and stores it.
37          $item->setCustomPrice($this->specialPrice);
38          $item->setOriginalCustomPrice($this->specialPrice);
39      }
40  }
```

- **Line 2** – `declare(strict_types=1)`: scalar types are enforced, so e.g. the `string $sku` really is a string.
- **Lines 6–9** – imports: `Observer` and `ObserverInterface` from the event framework, plus `Product` and `Quote\Item` for type hints.
- **Line 16** – `implements ObserverInterface` is required. The invoker only calls `execute()` on classes implementing it (developer mode throws otherwise).
- **Lines 18–22** – constructor property promotion with `readonly` (PHP 8.1+). The values come from `di.xml`. They are `readonly` because observers are shared instances by default, and the settings must not change between dispatches.
- **Line 24** – `execute(Observer $observer): void`, the single method of the interface. It returns nothing; an observer acts by changing the objects it receives.
- **Line 27** – `$observer->getEvent()->getData('product')` reads the `product` key from the array passed to `dispatch()`. (`$observer->getEvent()->getProduct()` would also work via magic getters.)
- **Lines 28–30** – **guard clause**: if it isn't our SKU, return immediately. Every add-to-cart in the store runs this observer, so the common path must be cheap.
- **Line 33** – reads `quote_item`, the cart line just created or updated.
- **Line 37** – `setCustomPrice()` – the price the totals use for this line (`Quote\Item::getCalculationPrice()` prefers the custom price over the catalog price).
- **Line 38** – `setOriginalCustomPrice()` – the same value for the “original price” side of the item (`getCalculationPriceOriginal()` / `getBaseCalculationPriceOriginal()`). Core's admin custom-price code, `Quote\Item\Updater::setCustomPrice()`, sets both fields to the same value, so we do too.
- **No save call** – the observer only changes the object in memory. `AddProductToCart` calls `Cart::save()` right after `addProduct()` returns, which collects totals with the custom price and stores the line. Saving here would duplicate that work.

6. Tested scenarios

Tested end-to-end over HTTP against the local storefront (`http://localhost:8080`) as a guest, the same way a browser does: form key, cookies and the add-to-cart controller. The cart page was then read and the stored quote checked in the database.

#	Action	Result
1	Add Realme XT (catalog $349.99)	“You added Realme XT to your shopping cart.” Cart line shows **$99.00**.
2	Add Oppo F19 Pro Plus 5G	Normal price **$399.99** – observer returned early.
3	Change Realme XT quantity to 2 on the cart page	Still **$99.00** each, row total $198.00.
4	Check stored totals	`quote_item`: price 99.00, custom_price 99.00, row_total 198.00. Quote subtotal and grand total **$597.99** (198.00 + 399.99).
5	Open the Realme XT product page	Still **$349.99** – the catalog price is untouched.

7. Install, configure and test

Install

terminal — from the Magento root

```
php bin/magento module:enable Codilar_EventDemo
php bin/magento setup:upgrade          # registers the module
php bin/magento setup:di:compile       # production mode only
php bin/magento cache:flush            # events.xml and di.xml are cached config
```

Configure

- **Different product or price** – edit the two `<argument>` values in `etc/frontend/di.xml`, then `bin/magento cache:flush`.
- **Switch the observer off** without deleting code – in a module that loads later: `<event name="checkout_cart_product_add_after"><observer name="codilar_eventdemo_cart_special_price" disabled="true"/></event>`
- **Remove the module** – `module:disable Codilar_EventDemo`, delete the folder, `setup:upgrade`, `cache:flush`.

Manual test script

1. Open Realme XT – the product page shows $349.99.
2. Click Add to Cart, then open the cart – the line shows $99.00.
3. Add any other product – it shows its normal price.
4. Change Realme XT's quantity to 2 – $99.00 each, row total $198.00.

Useful checks

terminal

```
php bin/magento module:status Codilar_EventDemo
# every module observing this event
grep -rl "checkout_cart_product_add_after" app/code vendor/magento --include=events.xml
# where it is dispatched
grep -rn "'checkout_cart_product_add_after'" vendor/magento --include=*.php
```

To confirm the observer runs, add a temporary log line inside `execute()`, or enable the profiler: it records `EVENT:` and `OBSERVER:` timers, as the dispatch code in section 2 shows.

8. Design decisions and pitfalls

Why an observer and not a plugin?

Magento already announces the exact moment we need, with exactly the data we need (the new cart line and the product). Reacting to it is the least intrusive option: no method signatures are involved and other observers keep working. A plugin would be needed only to change `addProduct()`'s arguments or result, or to stop it.

Why a custom price instead of `setPrice()`?

`price` on the quote item is recalculated from the catalog every time totals are collected, so a value set with `setPrice()` would be overwritten on the next save. The *custom price* is the field the totals collectors respect and store (`custom_price` and `original_custom_price` columns).

Why not change the product's price?

Changing `$product->setPrice()` would affect only this request's object and would be lost on recalculation. Saving it would change the catalog price for everyone. The requirement is a price for this cart line only.

No save in the observer

The event fires in the middle of the add-to-cart process, and `Cart::save()` follows straight after. Saving the quote inside the observer would duplicate work and could save half-finished data.

Storefront scope

The event is only dispatched by the storefront cart model, so registering it globally would add nothing but a config lookup in every other area. `etc/frontend` also documents intent: this is a storefront rule.

Living alongside Codilar_CartPlugins

- Plugins on `addProduct()` run **before** the event. If the guest or duplicate check throws, the event never fires, so a blocked add never gets the special price.
- The free phone cover is added with `Quote::addProduct()` directly, not `Cart::addProduct()`, so this event does not fire for it and the gift keeps its $0 price.

Common observer mistakes

- Forgetting `implements ObserverInterface` – execute() is never called (exception in developer mode).
- A typo in the event name – no error at all, the observer just never runs.
- Putting events.xml in the wrong area folder – works in one area, silently missing in another.
- Not flushing the cache after editing events.xml – the old configuration keeps running.
- Doing heavy work (API calls, full loads, saves) in observers of frequent events – it slows every request that fires them.
- Relying on observer order – Magento does not guarantee the order between observers of the same event.

9. Known limitations

- **Only at add time** – the price is set when the product is added. Lines already in carts before the module was enabled keep the catalog price until removed and re-added.
- **Survives configuration changes** – the custom price is stored on the quote line. Changing the price in di.xml, or disabling the module, does not update lines already in carts.
- **Not on every path** – reorder, admin orders, REST and GraphQL do not fire this event (section 3). Enforcing the price everywhere would need a different hook, e.g. the `sales_quote_collect_totals_before` event or a quote totals collector.
- **Overrides other catalog pricing** – a custom price replaces special prices, tier prices and catalog price rules for that line. Cart price rules (coupons, discounts) still apply on top.
- **One SKU** – a single product and price. Several products would need an array argument (`xsi:type="array"`) or an admin setting.
- **Simple products** – built and tested for a simple product. For a configurable product, the SKU check and the line to price would need to account for the parent/child item pair.

10. Expected questions and answers

Events and observers in general

Q1. What is an event in Magento 2?

A named announcement that code dispatches with `EventManager::dispatch($name, $data)` at a meaningful moment, carrying data such as the objects involved. Any module can react to it without the dispatcher knowing.

Q2. What is an observer?

A class implementing `Magento\Framework\Event\ObserverInterface`. Its `execute(Observer $observer)` method runs whenever an event it is registered for (in events.xml) is dispatched.

Q3. Which design pattern is this?

Publish–subscribe, a form of the Observer pattern. The dispatcher (publisher) and observers (subscribers) are decoupled through the event manager and configuration.

Q4. How do you register an observer?

In `etc/[area/]events.xml`: `<event name="event_name"><observer name="unique_name" instance="Vendor\Module\Observer\Class"/></event>`, then flush the cache.

Q5. What is the difference between `etc/events.xml` and `etc/frontend/events.xml`?

`etc/events.xml` is global and applies in every area. `etc/frontend`, `etc/adminhtml`, `etc/webapi_rest`, `etc/graphql` and so on apply only to that area. This demo uses `frontend` because the rule is for storefront shoppers.

Q6. How do you read the event data inside an observer?

`$observer->getEvent()->getData('key')` or the magic getter `$observer->getEvent()->getKey()`. The keys are those of the array passed to `dispatch()`. Here they are `quote_item` and `product`.

Q7. Can an observer return a value or change the result of the dispatching method?

No. `execute()` returns `void`. An observer acts by changing the objects passed in the event data (as we do with the quote item) or by doing side work. To change a method's arguments or return value, use a plugin.

Q8. Can an observer stop the process?

Only by throwing an exception, which the calling code may or may not handle. It cannot cleanly skip the dispatching method the way an around plugin can.

Q9. In which order do observers of the same event run?

In the order the merged configuration lists them, which depends on module load order. Magento does not guarantee or let you set an order (there is no sortOrder for observers), so observers must not depend on each other.

Q10. What do `disabled` and `shared` do?

`disabled="true"` switches an observer off; it's used with the same event and observer name to disable another module's observer. `shared="false"` makes Magento create a new observer instance per dispatch instead of reusing one shared instance.

Q11. What happens if the observer does not implement `ObserverInterface`?

The invoker does not call it. In developer mode it throws a `LogicException`; in production it logs a warning.

Q12. Can you use dependency injection in an observer?

Yes. Observers are created by the object manager, so constructor injection works, including scalar arguments from di.xml as in this demo.

Q13. How do you create your own event?

Inject `Magento\Framework\Event\ManagerInterface` and call `$this->eventManager->dispatch('vendor_module_something', ['key' => $value])`. Others can then observe `vendor_module_something`.

Q14. What are model events such as `catalog_product_save_after`?

Every model extending `AbstractModel` dispatches `{eventPrefix}_load_after`, `_save_before`, `_save_after`, `_delete_before` and so on, using its `$_eventPrefix` property. The object is available under `$_eventObject` (e.g. `product`) or `data_object`.

Q15. Observer, plugin or preference: how do you choose?

Use an observer if a suitable event exists and you only need to react. Use a plugin to change a public method's input, output or execution. Use a preference only to replace a whole class.

Q16. Why are observers on frequent events risky?

They run on every dispatch, often many times per request. Heavy work there (saving entities, external API calls) slows the whole store. Keep a cheap guard clause first, as this observer does with the SKU check.

About this demo

Q17. Which event does the demo use, and where is it dispatched?

`checkout_cart_product_add_after`, dispatched at the end of `Magento\Checkout\Model\Cart::addProduct()` (Cart.php lines 419–422 in 2.4.9) with `quote_item` and `product`.

Q18. Why this event and not `sales_quote_add_item` or `checkout_cart_add_product_complete`?

`checkout_cart_product_add_after` passes the finished cart line together with the product, after the add succeeded and before the cart is saved. That is exactly when a custom price can be set and saved for free. `checkout_cart_add_product_complete` fires in the controller after the cart has already been saved, so we would have to save again.

Q19. How does the observer pick the product?

It compares `$product->getSku()` with the `sku` argument from di.xml (`realme-xt`) and returns immediately for any other product.

Q20. Why `setCustomPrice()` and not `setPrice()`?

The quote item's `price` is recalculated from the catalog whenever totals are collected, so it would be overwritten. The custom price is what the totals use for that line, and it is stored in the `custom_price` column.

Q21. Why set both `custom_price` and `original_custom_price`?

`Quote\Item::getCalculationPrice()` reads `custom_price`, while `getCalculationPriceOriginal()` and `getBaseCalculationPriceOriginal()` read `original_custom_price`. Core's admin custom-price code (`Quote\Item\Updater::setCustomPrice()`) sets both to the same value, so every price calculation on the line sees $99.

Q22. Core's admin code also calls `$item->getProduct()->setIsSuperMode(true)`. Why doesn't this observer?

In 2.4.9 nothing reads that product-level flag. Core only checks super mode on the **quote** (`$quote->getIsSuperMode()`, used by the subtotal collector and the stock quantity validator for admin orders). The line would have no effect, so it was left out, and setting super mode on the whole storefront quote would wrongly relax stock checks.

Q23. Why doesn't the observer save the quote?

The add-to-cart service calls `Cart::save()` immediately after `addProduct()`. That collects totals with the custom price and saves the line. Saving in the observer would do the work twice.

Q24. Does the catalog price change?

No. Only the cart line has the special price. The product page still shows $349.99 (verified in testing).

Q25. What happens when the shopper changes the quantity?

The event does not fire for quantity updates, but the custom price is stored on the line, so every unit stays $99. In testing, quantity 2 gave a $198 row total.

Q26. How do you change the product or price?

Edit the `sku` and `specialPrice` arguments in `etc/frontend/di.xml` and run `bin/magento cache:flush`. The object manager passes them to the constructor by parameter name.

Q27. Did you change anything in the admin?

No admin code, configuration or product data. The special price exists only on the cart line.

Q28. Does it work for admin orders, the REST API or GraphQL?

No. Those paths do not use `Checkout\Model\Cart`, so the event isn't dispatched, and the observer is registered for the storefront only. Applying the price everywhere would need a hook on quote totals collection instead.

Q29. How does it interact with the cart plugins module?

The plugins on `addProduct()` run first. A blocked add (guest or duplicate) never reaches the event. The free gift is added via `Quote::addProduct()`, which doesn't dispatch this event, so the gift is unaffected.

Q30. Will a coupon still apply on top of $99?

Yes. Cart price rules are applied by the discount collector after the item price is known, so they discount the $99 line normally. Catalog-level pricing (special price, tier price, catalog rules) is replaced by the custom price.

Q31. What happens to existing carts if the module is disabled?

Lines already in carts keep $99, because the custom price is stored on the quote item. Only new adds get the catalog price.

Q32. How was it tested?

End-to-end over HTTP against the storefront as a guest: add Realme XT, add another product, change quantity, then check the cart page, the stored quote totals ($597.99) and the product page price (section 6).

Q33. How can you see every observer of this event?

`grep -rl "checkout_cart_product_add_after" app/code vendor/magento --include=events.xml`. Here that is Magento_Quote's `setBasePriceObserver` and our `codilar_eventdemo_cart_special_price`.